

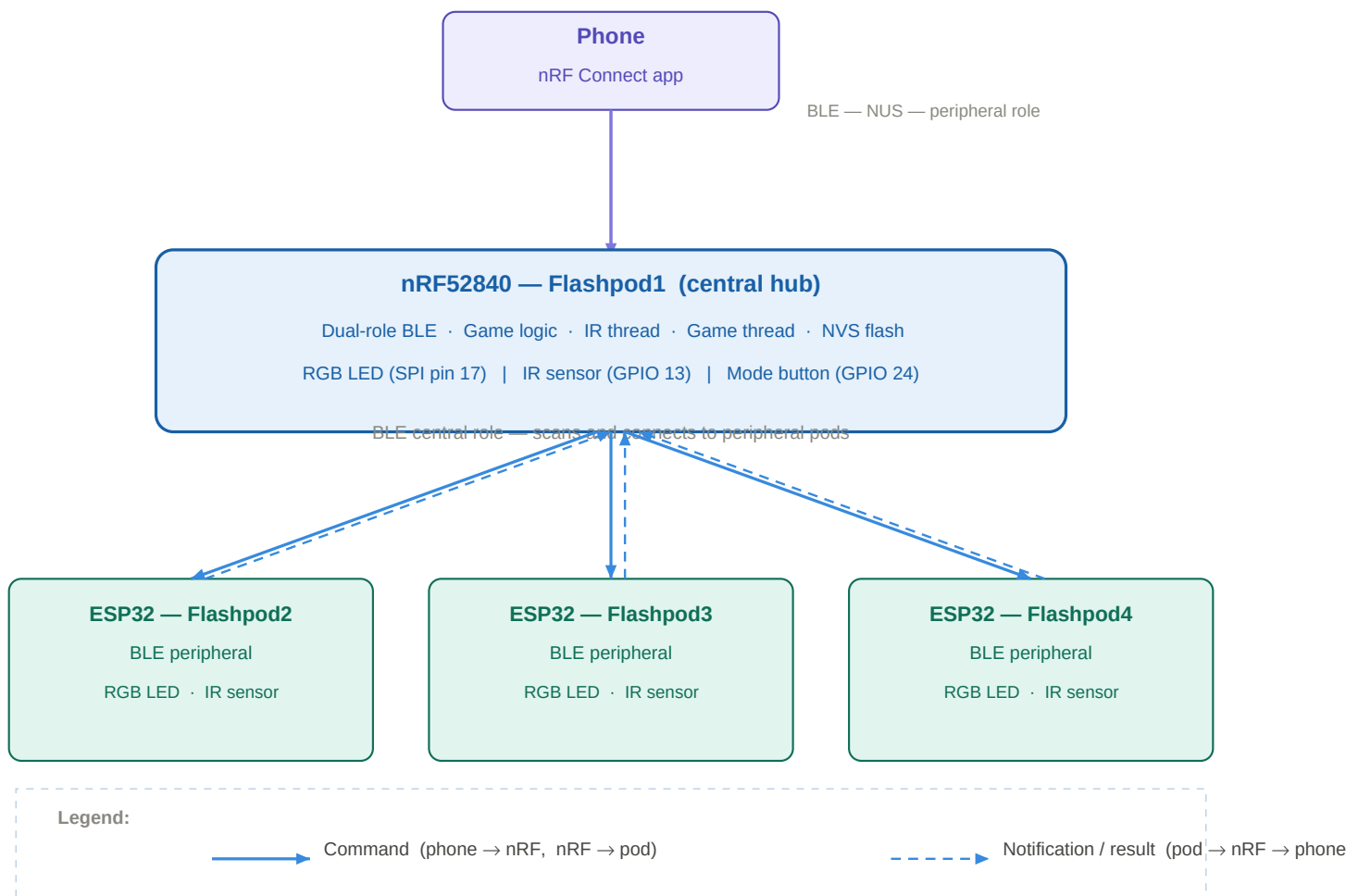
Flashpod

Cognitive Strength & Coordination System

Objective

Flashpod is a wireless, IR-based reaction training system designed to improve cognitive processing speed and neuromuscular coordination. A central nRF52840 pod manages three ESP32 peripheral pods over Bluetooth Low Energy, while a smartphone is used to configure and monitor the session. Two game modes challenge the user: Mode 1 activates a single random pod; Mode 2 lights all four pods with one showing a different colour, requiring the user to identify and tap the odd pod. Reaction times are measured in milliseconds and reported to the phone in real time.

Architecture Diagram



Key Features

1 Non-contact IR proximity triggering

Hand detection via active-low GPIO without any physical contact. The IR sensor output goes LOW when a hand is within range, triggering the reaction time calculation immediately.

2 High-precision reaction timing

Reaction time is calculated in milliseconds from the moment the LED turns on to the moment the IR sensor detects the user's hand. Results are sent to the phone via BLE notification in real time.

3 Two cognitive game modes

Mode 1: a single random pod lights up in a colour chosen by the phone. Mode 2 (odd one out): all four pods light up simultaneously — three in one colour and one in a different colour. The user must identify and tap the odd-coloured pod. Correct tap reports RT; wrong tap reports WRONG.

4 Asynchronous BLE peripheral management

The nRF52840 operates in dual BLE role simultaneously — peripheral to the phone and central to three ESP32 pods. Scan interval is tuned to 800ms/50ms window so the radio can advertise to the phone between scans.

5 Configurable timeout with NVS persistence

The phone sends TIMEOUT:x (seconds) via NUS. The nRF saves the value to NVS flash so it survives power cycles. The new timeout is also forwarded to all ESP32 pods so the whole system stays in sync.

6 Mid-game mode switching via GPIO button

A push button on GPIO pin 24 toggles between Game Mode 1 and Mode 2 at any time, including during an active game. The ISR signals round_done_sem to cleanly end the current round and game_start_sem to immediately restart in the new mode.

7 Zephyr RTOS multithreading

Three concurrent threads: IR thread (priority 5) polls the sensor every 50ms; Game thread (priority 6) manages round logic and pod selection; Main thread handles BLE advertising. Semaphores coordinate between threads and mutexes protect all shared state variables.

8 Mechanical wear reduction

No physical button press required — IR proximity sensing only. This eliminates mechanical contact, reducing hardware wear and enabling the system to be used in high-repetition training sessions.

Hardware

nRF52840 (Flashpod1 — central)

Role:	BLE dual-role: peripheral to phone, central to ESP32 pods
LED output:	WS2812B RGB strip via SPI1 (MOSI: pin 17, SCK: pin 31)
IR sensor:	Active-low proximity sensor on GPIO 13
Mode button:	Push button on GPIO 24 (active-low, pull-up)
Flash storage:	NVS on internal flash — stores timeout across reboots

ESP32 (Flashpod2 / 3 / 4 — peripheral)

Role:	BLE peripheral — receives commands, sends results to nRF
LED output:	WS2812B RGB strip via GPIO 16 (NeoPixel library)
IR sensor:	Active-low proximity sensor on GPIO 13
Timeout:	Received from nRF via BLE command TIMEOUT:ms

Software Stack

nRF52840 firmware

RTOS:	Zephyr RTOS via nRF Connect SDK v3.2.4
BLE stack:	SoftDevice Controller — bt_le_adv, bt_le_scan, bt_gatt
NUS service:	bt_nus_send / bt_nus_cb_register for phone communication
LED driver:	WS2812B via Zephyr LED strip API + SPI
Storage:	Zephyr NVS — nvs_mount, nvs_read, nvs_write
Threading:	k_thread_create, k_sem_give/take, k_mutex_lock/unlock
GPIO:	gpio_pin_configure_dt, gpio_pin_interrupt_configure_dt

ESP32 firmware

Framework:	Arduino framework with FreeRTOS
BLE:	ESP32 BLE Arduino library — BLEServer, BLECharacteristic
LED:	Adafruit NeoPixel library
IR task:	FreeRTOS task — xTaskCreatePinnedToCore, vTaskDelay
Concurrency:	portENTER_CRITICAL / portEXIT_CRITICAL for shared state

BLE Communication Flow

Phone → nRF:	RED / GREEN / BLUE / YELLOW / PINK / VIOLET → start Mode 1 with chosen colour
Phone → nRF:	START → start Mode 2 STOP → stop game TIMEOUT:x → set timeout
nRF → ESP32:	ON:RRGGBB → light LED with hex colour OFF → turn off LED TIMEOUT:ms → sync timeout
ESP32 → nRF:	PODx:RT:ms → reaction time PODx:TIMEOUT → timed out
nRF → Phone:	PODx:RT:ms → result PODx:WRONG → wrong pod tapped PODx:TIMEOUT